

SPARCS Hospital Inpatient Discharges

Chen ER/EER Design, Relationship Testing, Cleaning, and MySQL Implementation Report

Dataset: New York State Hospital Inpatient Discharges (SPARCS De-Identified), 2021.

Purpose

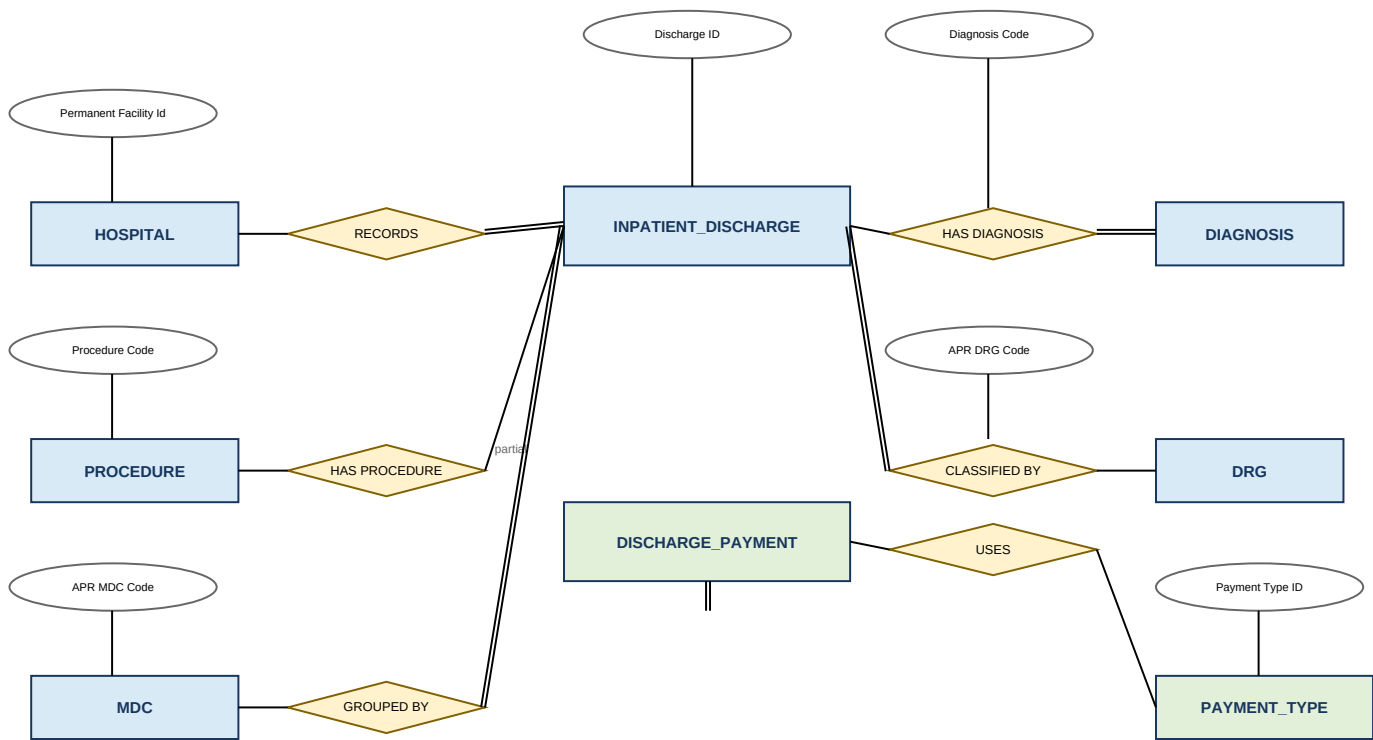
This report converts the 33-column flat file into a proposed relational design. Relationships should be tested with SQL before final tables are created. The design distinguishes conceptual participation from source missing values caused by de-identification or incomplete records.

Main decisions

- One row represents one inpatient discharge, not a uniquely identifiable patient.
- DISCHARGE_PAYMENT is the associative entity for Payment Typology 1, 2, and 3.
- Diagnosis, procedure, DRG, and MDC code-description tables are reference entities only after functional-dependency tests confirm code determines description.
- Demographics remain discharge attributes because no patient identifier exists.
- Use double-line total participation only when the business rule is confirmed; source nulls may require nullable MySQL foreign keys.

1. Chen-Notation ER Diagram

Rectangles are entities, diamonds are relationships, and ovals are attributes. Double lines indicate total participation; single lines indicate partial participation.



The complete 33-column mapping is shown below. DISCHARGE_PAYMENT is highlighted as the associative entity.

2. Complete 33-Column Mapping

Entity/table	Original columns represented
HOSPITAL	Hospital Service Area; Hospital County; Operating Certificate Number; Permanent Facility Id; Facility Name
INPATIENT_DISCHARGE	Age Group; Zip Code - 3 digits; Gender; Race; Ethnicity; Length of Stay; Type of Admission; Patient Disposition; Discharge Year; APR Severity of Illness Code; APR Severity of Illness; APR Risk of Mortality; APR Medical Surgical; Birth Weight; Emergency Department Indicator; Total Charges; Total Costs
DIAGNOSIS	CCSR Diagnosis Code; CCSR Diagnosis Description
PROCEDURE	CCSR Procedure Code; CCSR Procedure Description
DRG	APR DRG Code; APR DRG Description
MDC	APR MDC Code; APR MDC Description
DISCHARGE_PAYMENT	Payment Typology 1; Payment Typology 2; Payment Typology 3 represented as payment_order 1, 2, and 3

The 33 columns are grouped as 5 hospital, 18 discharge, 2 diagnosis, 2 procedure, 2 DRG, 2 MDC, and 3 payment columns.

3. Relationship and Participation Decisions

Relationship	Cardinality	Decision
HOSPITAL - DISCHARGE	1:N	Hospital records many discharges. Discharge should participate totally, but source hospital identifiers may be missing.
DIAGNOSIS - DISCHARGE	1:N	Confirm diagnosis code completeness and code-description dependency.
PROCEDURE - DISCHARGE	1:0..N	Partial because procedure code may be blank.
DRG - DISCHARGE	1:N	Confirm every discharge has an APR DRG code.
MDC - DISCHARGE	1:N	Confirm every discharge has an APR MDC code.
DISCHARGE - DISCHARGE_PAYMENT	1:N	One discharge has one to three payment rows.
PAYMENT_TYPE - DISCHARGE_PAYMENT	1:N	One payment type can be used by many discharge-payment rows.

4. Data Cleaning and Decision Table

Run each check on the raw staging table before creating final tables. Record the result and use it to choose data types, NULL rules, keys, and foreign keys.

Step	Simple SQL	Decision
Confirm row grain	SELECT COUNT(*) FROM raw_inpatient_discharges;	One row should represent one discharge; do not call discharge_id a patient ID.
Convert blanks to NULL	UPDATE raw_inpatient_discharges SET zip_code_3_digits=NULL WHERE TRIM(zip_code_3_digits)=""	Blank strings should become NULL.
Check missingness	SELECT COUNT(*), COUNT(ccsr_procedure_code), COUNT(payment_typology_2), COUNT(payment_typology_3) FROM raw_inpatient_discharges;	Use nullable columns where missingness is legitimate.
Validate ZIP	SELECT DISTINCT zip_code_3_digits FROM raw_inpatient_discharges ORDER BY 1;	Keep ZIP as VARCHAR(3), not INT.
Validate numbers	SELECT COUNT(*) FROM raw_inpatient_discharges WHERE total_charges REGEXP '[^0-9.]' OR total_costs REGEXP '[^0-9.]';	Clean currency text and cast to DECIMAL.
Hospital dependency	SELECT permanent_facility_id, COUNT(DISTINCT facility_name) n FROM raw_inpatient_discharges GROUP BY permanent_facility_id HAVING n>1;	Zero rows support facility ID -> facility name.
Diagnosis dependency	SELECT ccsr_diagnosis_code, COUNT(DISTINCT ccsr_diagnosis_description) n FROM raw_inpatient_discharges GROUP BY ccsr_diagnosis_code HAVING n>1;	Zero rows support diagnosis reference table.
DRG dependency	SELECT apr_drg_code, COUNT(DISTINCT apr_drg_description) n FROM raw_inpatient_discharges GROUP BY apr_drg_code HAVING n>1;	Zero rows support DRG(code, description).
MDC dependency	SELECT apr_mdc_code, COUNT(DISTINCT apr_mdc_description) n FROM raw_inpatient_discharges GROUP BY apr_mdc_code HAVING n>1;	Zero rows support MDC(code, description).
Payment repeating group	SELECT COUNT(*), COUNT(payment_typology_1), COUNT(payment_typology_2), COUNT(payment_typology_3) FROM raw_inpatient_discharges;	Unpivot payment columns into DISCHARGE_PAYMENT.
Financial values	SELECT COUNT(total_charges), COUNT(total_costs) FROM raw_inpatient_discharges;	Use DECIMAL(14,2); allow NULL when absent.
Severity dependency	SELECT apr_severity_of_illness_code, COUNT(DISTINCT apr_severity_of_illness), COUNT(DISTINCT apr_risk_of_mortality) FROM raw_inpatient_discharges GROUP BY apr_severity_of_illness_code;	Do not make one combined lookup unless dependency is confirmed.

5. Normalized Relational Schema

```
HOSPITAL(hospital_id PK, hospital_service_area, hospital_county,  
         operating_certificate_number, permanent_facility_id, facility_name)  
  
INPATIENT_DISCHARGE(discharge_id PK, hospital_id FK, ccsr_diagnosis_code FK,  
                   ccsr_procedure_code FK NULL, apr_drg_code FK, apr_mdc_code FK, age_group,  
                   zip_code_3_digits, gender, race, ethnicity, length_of_stay, type_of_admission,  
                   patient_disposition, discharge_year, apr_severity_of_illness_code,  
                   apr_severity_of_illness, apr_risk_of_mortality, apr_medical_surgical,  
                   birth_weight, emergency_department_indicator, total_charges, total_costs)  
  
DIAGNOSIS(ccsr_diagnosis_code PK, ccsr_diagnosis_description)  
PROCEDURE(ccsr_procedure_code PK, ccsr_procedure_description)  
DRG(apr_drg_code PK, apr_drg_description)  
MDC(apr_mdc_code PK, apr_mdc_description)  
PAYMENT_TYPE(payment_type_id PK, payment_type_name)  
DISCHARGE_PAYMENT(discharge_id PK/FK, payment_type_id PK/FK, payment_order)
```

6. Why this design is in 3NF

Hospital attributes are separated because hospital_id determines hospital details. Code descriptions are separated because each code should determine one description. Payment Typology 1, 2, and 3 are transformed into rows because they form a repeating group. DISCHARGE_PAYMENT has a composite key and preserves payment order.

7. SQL Validation Queries

```
USE sparcs_db;

SELECT COUNT(*) AS total_rows
FROM raw_inpatient_discharges;

SELECT COUNT(*) AS total_rows, COUNT(ccsr_procedure_code) AS procedure_present,
       COUNT(payment_typology_2) AS payment_2_present,
       COUNT(payment_typology_3) AS payment_3_present
FROM raw_inpatient_discharges;

SELECT ccsr_diagnosis_code, COUNT(DISTINCT ccsr_diagnosis_description) AS descriptions
FROM raw_inpatient_discharges GROUP BY ccsr_diagnosis_code HAVING descriptions > 1;

SELECT apr_drg_code, COUNT(DISTINCT apr_drg_description) AS descriptions
FROM raw_inpatient_discharges GROUP BY apr_drg_code HAVING descriptions > 1;

SELECT apr_mdc_code, COUNT(DISTINCT apr_mdc_description) AS descriptions
FROM raw_inpatient_discharges GROUP BY apr_mdc_code HAVING descriptions > 1;
```

8. Lecture Concepts Applied

Concept	Application
Strong entity	Hospital, discharge, diagnosis, procedure, DRG, MDC, and payment type have identifying keys.
Associative entity	DISCHARGE_PAYMENT resolves the many-to-many discharge-payment relationship.
Total participation	Double line from discharge when every discharge must participate in a relationship.
Partial participation	Single line for optional procedure information and redacted source fields.
Weak entity	Not required because no entity has only a partial key owned by another entity.
Specialization/generalization	Not required unless the project introduces subclasses.
Category/union	Not supported because there is no union of different superclasses.
Aggregation	Not required because no relationship must participate in another relationship.

Conclusion

This is a defensible Chen ER design for the available data. Final foreign-key nullability and total-participation lines should be finalized after the SQL tests are executed. The de-identified source limits patient-level analysis, so the model represents discharges rather than identifiable patients.

Source

New York State Health Data: Hospital Inpatient Discharges (SPARCS De-Identified).
https://health.data.ny.gov/Health/Hospital-Inpatient-Discharges-SPARCS-De-Identified/tg3i-cinn/about_data